

WORKSHEET 5: ENVIRONMENTS AND CONTAINERIZATION

Kosmas Hench
2025-05-28

CONTENTS

1	Environments	1
1.1	In Action	2
1.2	Infrastructure	4
1.3	Installation and Management .	5
2	Container	7
2.1	In Use	8
2.2	Import and Inspection	8
3	Workflow Integration	9

In this SESSION, we will conclude the workshop by looking at techniques that allow to share an entire working environment. This allows others to rerun code under the same conditions as you, and it guards your work against breaking changes that can happen when updating software in the middle of an analysis. We will demonstrate what can happen when code and software are a mismatch, and finally we will see how we can incorporate the specification of a precise working environment when using a workflow manager.

1. ENVIRONMENTS

As preparation for today's practical part, we set up another small project called `environment_demo`. We start out by creating the folder and establishing our default project structure (`data/` `code/` `results/`).

```
mkdir environment_demo # create project folder
cd environment_demo
mkdir -p data code/R code/py results/img/ # create folder structure
```

We will re-use the penguin data set from SESSION 1, which we import alongside some R and python scripts from the `external_resources` folder in your `de.NBI $HOME` directory.

```
cp ~/external_resources/data/penguins.tsv data/ # import data file
cp ~/external_resources/scripts/R/environment_demo.Rproj ./ # init RStudio project
cp ~/external_resources/scripts/R/script* code/R/ # import R scripts
cp ~/external_resources/scripts/py/demo* code/py/ # import python scripts
```

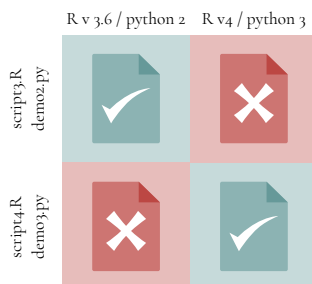
This should provide us with the following setup for today:

```
tree
```

```
├── code/
│   ├── R/
│   │   ├── script3.R
│   │   └── script4.R
│   └── py/
│       ├── demo2.py
│       └── demo3.py
├── data/
│   └── penguins.tsv
├── environment_demo.Rproj
├── results/
│   └── img/
```

1.1 In Action

To demonstrate the need for encapsulated working environments, let's look at some examples where things go wrong. In the following, we are going to look at a set of scripts that contain working code – but we will make them fail by executing them using the wrong software versions:



We will be able to specify the software versions that are used for the code execution by running the scripts within `conda` environments. These contain isolated installations of `R` or `python` and allow matching software and code.

R examples

We start with the `R`-based example. As part of the project setup, we copied two scripts into our `code/R` folder: `script3.R` was written in `Rv3.6`, while `script4.R` was written in `Rv4`.

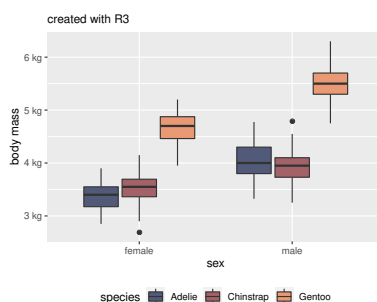
Often, the version numbers of software indicate degree of change during an update. This means that a change from `Rv3.6.3` to `Rv4.0.0` is a very different beast than for example an update from `Rv4.4.2` to `Rv4.4.3`.

→ The change in the first number indicates that the first type of update comes with more severe changes in how the code works. This update has much more potential to introduce breaking changes, and code written for older versions will likely behave differently in newer versions. This is exactly what we are going to look at.

So, now let's first run the `script3.R` as it is intended. For this, we prefix the usual command `Rscript <scriptname.R>` with `conda run -n <environment_name>`. This will force the command to be executed within the specified `conda` environment (which in this case contains the right version: `Rv3.6.2`).

```
conda run -n r3_tidy Rscript code/R/script3.R
```

The code should run successfully and produce the file `results/img/plot_r3.png`:



However, the same code will produce an error and fail when executed using a different `R` version (the `conda` environment `r4_tidy` contains `Rv4.4.3`):

```
conda run -n r4_tidy Rscript code/R/script3.R
```

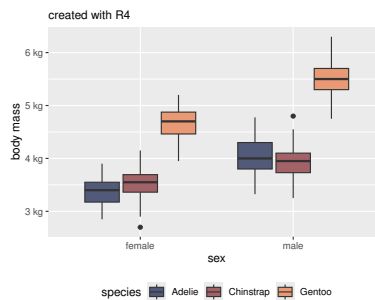
```
Error in `palette()`:  
! Insufficient values in manual scale. 3 needed but only 1 provided.
```

The code will still create (overwrite) the file `results/img/plot_r3.png`, but now it is an empty plot.

To create a similar plot with `Rv4`, the code needs to be slightly re-written as was done in `script4.R`.

```
conda run -n r4_tidy Rscript code/R/script4.R
```

Again, this should run successfully and produce `results/img/plot_r4.png`:



However, `script4.R` fails when trying to run it in the conda environment `r3_tidy`:

```
conda run -n r3_tidy Rscript code/R/script4.R
```

```
Error: unexpected ">" in "data <- read.delim(here("data/penguins.tsv"), sep = "\t") >"
Execution halted
```

python examples

Breaking changes caused by software updates are a general topic that are not limited to R. Similar issues arise in python where a major transition happened with the shift from python v2 to python v3.

So, similarly to the R example, also python code will behave strangely when executed using a different version than the one it was written in. In the following we will execute the scripts `demo2.py` and `demo3.py` using the two conda environments `py2_base` and `py3_base`. For both scripts, the task is simple to take a user-specified number, divide it by 2 and print the results to the screen.

When successfully running `demo2.py`, it will report the result as *integer*, so it will round the result downwards (default behavior of python v2 in this case).

```
conda run -n py2_base python code/py/demo2.py 5
```

```
Half of your number is: 2
```

Using the wrong python version will casuse the same script to fail:

```
conda run -n py3_base python code/py/demo2.py 5
```

```
SyntaxError: Missing parentheses in call to print. Did you mean print(...)?
```

When implementing the same idea in python v3, there is a subtle change in behavior, as now the result is returned as float (it will not be rounded).

```
conda run -n py3_base python code/py/demo3.py 5
```

```
Half of your number is: 2.5
```

If we execute `demo3.py` with python v2, we actually get the result, but the formatting is jumbled up:

```
conda run -n py2_base python code/py/demo3.py 5
```

```
("Half of your number is:", 2)
```

→ The point here is that things can go haywire if code is run with a different software version of than the one that it was written for. The really critical issues are those where scripts don't fail completely, but the output changes slightly. These cases can easily go undetected. Using conda environments we can execute each script in a fitting environment that takes the required software versions into consideration.

1.2 Infrastructure

In the examples above, we have seen that we can use `conda` environments to switch which version of `R` or `python` is being called when executing a script.

Now, let's try to understand what is happening under hood when we toggle different `conda` environments. A key element to understand what is going on when using `conda` is the so called `$PATH` – a variable used by your operating system to locate “executable files” (i.e. installed software).

All operating systems (unix derived like linux or mac, but also windows) use a variable called `$PATH`, and on all it describes a list of directories within the operating system. This list of directories describes where to locate the programs that are installed, and also introduce a hierarchy (in which folder to look first, and which program to prefer in case of a duplication). So, when we type `Rscript` into the console, or double-click the `Rstudio` icon, the operating system will work through that list of directories trying to find the requested program:



This is where `conda` intercepts, and re-directs the searching behavior. To inspect this, let's first deactivate `conda` to see how the operating system is operating without `conda` interfering.

```
cd
conda deactivate # 2x, until there is no (<env_name>) tag at the start of the prompt
```

In this state, we can look at the default `$PATH` of the operating system:

```
echo $PATH
/usr/lib/code-server/lib/vscode/bin/remote-cli:/home/ubuntu/.local/bin:/home/ubuntu/
miniforge/bin:/home/ubuntu/miniforge/condabin:/usr/local/sbin:/usr/local/bin:/usr/
sbin:/usr/bin:/snap/bin
```

As expected, we are presented with a list of directories (separated by a “:”) pointing to several locations within the guts of the system.

Now, let's see what happens once we start `conda`:

```
conda activate # → (base) $
echo $PATH
/home/ubuntu/miniforge/bin:/usr/lib/code-server/lib/vscode/bin/remote-cli:/home/ubuntu/.
local/bin:/home/ubuntu/miniforge/bin: ...
```

As you can see, `conda` will prepend the path, tagging a directory that is managed by `conda` to the start of the `$PATH`. That way, any software that is installed into this `conda` directory will be preferred over other versions of the same program (because the operating system works through the `$PATH` from front to back on its search for a program).

In this case the folder managed by `conda` is `/home/ubuntu/miniforge`. Usually this folder is located in the `$HOME` directory of a user (like in this case), but the exact naming of this folder might vary (depending on the `conda` installation).

When looking into this folder, we can see two subfolders of interest: `bin` and `envs`:

```
ls miniforge
LICENSE.txt _conda bin compiler_compat conda-meta condabin env-ymls envs etc fonts ...
```

→ the subfolder `/home/ubuntu/miniforge/bin` is the same one that was prepended to the `$PATH` above. This is the location where software installed into `conda`'s `base` environment are installed into.

So the `R` version that is executed when running the following command...

```
R --version
R version 4.4.2 (2024-10-31) -- "Pile of Leaves"
```

...is in fact the executable file “R” located in the subfolder `bin` of the `conda` directory...

```
which R
/home/ubuntu/miniforge/bin/R
```

...where it sits next to all other programs installed into the `conda` base environment:

```
ls miniforge/bin
2to3 2to3-3.12 R Rscript activate ...
```

However, we can have more environments than just “base”: in our example above, we made use of the four environments “`r3_tidy`”, “`r4_tidy`”, “`py2_base`” and “`py3_base`”.

We can easily get an overview of all currently available `conda` environments:

```
conda env list
# conda environments:
#
base          * /home/ubuntu/miniforge
denbi         /home/ubuntu/miniforge/envs/denbi
py2_base      /home/ubuntu/miniforge/envs/py2_base
py3_base      /home/ubuntu/miniforge/envs/py3_base
r3_tidy       /home/ubuntu/miniforge/envs/r3_tidy
r4_tidy       /home/ubuntu/miniforge/envs/r4_tidy
```

All environments other than “base” are living inside the `conda` subfolder `envs/`, which includes one subfolder per existing environment.

If we look into one of those, the content looks quite similar to that of the main `conda` folder — in particular, they all include their own `bin/` folder.

```
ls miniforge/envs/r4_tidy
bin cmake conda-meta etc fonts include lib libexec man sbin share ...
```

Similarly to the “base” environment, these are the location where installed software is stored for each environment:

```
conda run -n r4_tidy which R
/home/ubuntu/miniforge/envs/r4_tidy/bin/R
```

And when activating a particular `conda` environment, this directory is tagged to start of the `$PATH` variable. Therefore, the software installed into this directory is prioritized once the environment is activated.

```
conda activate r4_tidy # → (r4_tidy) $
echo $PATH
/home/ubuntu/miniforge/envs/r4_tidy/bin:/usr/lib/code-server/lib/vscode/bin/remote-cli:/home/ubuntu/.local/bin: ...
```

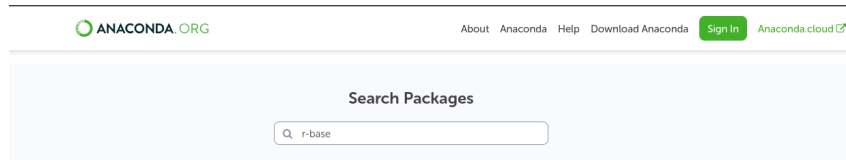
Finally, to get back to the default settings we need to deactivate the used environment.

```
conda deactivate # → (base) $
```

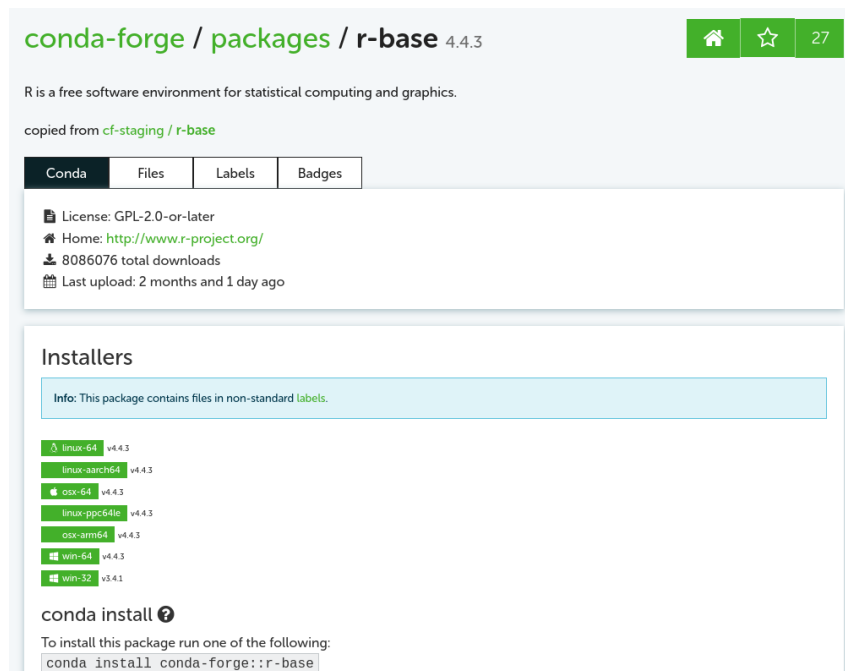
1.3 Installation and Management

Now that we know how to use them, let’s see how you can create `conda` environments that contain the specific software that you require for your own project.

In general there is a huge stack of software available and you can check which programs (and versions) are available at anaconda.org.



For all available software there is an overview of the most current version number, for which operating systems this software is available, and a one-liner that you can use to install this software into the currently active conda environment:



In this case the command to install R into the currently active environment would be `conda install conda-forge::r-base`.

While this approach does work, in the sense that you can install and run software on your computer that way, it is not what we are looking for when we are thinking of reproducibility:

To create a reproducible environment we want to use a recipe that we can share, which we store in the `envs/` folder of our `snakemake` workflow.

```
mkdir -p environment_demo/code/workflow/envs
cd environment_demo/code/workflow/envs
```

The recipes for conda this come in the form of a `yml` (or `yaml`) file, which list all required softwares. You can create a minimal `yml` file as follows:

```
echo "# conda env create -f yolo.yml
name: yolo
channels:
  - defaults
  - conda-forge
dependencies:
  - python
  - r-base" > yolo.yml
```

Then, we can use this `yml` file to create a pre-defined environment:

```
conda env create -f yolo.yml
```

Note, that we did not specify the software versions, so `conda` installs the current versions and what exactly is installed depends on the date that the environment is created.

The exact content of the environment is thus a little surprise:

```
conda run -n yolo R --version
```

```
R version 4.3.1 (2023-06-16) -- "Beagle Scouts"
```

Again, while this works to get software installed, it is still not optimal for reproducibility.

So, to make sure others can recreate the exact same environment, we can specify the exact version numbers in the `yml` file. This will always install the same content, even if newer versions of the required programs become available.

```
echo "# conda env create -f reproducible.yml
name: reproducible
dependencies:
  - conda-forge::python=3.13.3
  - conda-forge::r-base=4.4.3
  - conda-forge::pip=25.0.1
  - pip:
    - simplejson=3.17.0" > reproducible.yml
```

The creation of the environment remains exactly the same...

```
conda env create -f reproducible.yml
```

...but now, we get specifically the versions that we ordered.

```
conda run -n reproducible R --version
```

```
R version 4.4.3 (2025-02-28) -- "Trophy Case"
```

```
conda run -n reproducible pip list
```

Package	Version
pip	25.0.1
simplejson	3.17.0

Finally, if your computer becomes too crowded, it is easy to get rid of environments that you do not use:

```
conda env remove -n yolo
```

2. CONTAINER

The use of `conda` environments already goes a long way to ensure the reproducibility of an analysis. By providing the recipe for an environment, it is possible for others to recreate the exact working environment originally used – that is, provided that same infrastructure behind `conda` is maintained. People still need to be able to access the same software when re-building your environment.

An alternative to `conda` is the usage of containers. In contrast to `conda` (which is documenting the instructions to create an environment), containers bake the working environment into an actual file that you can share and/or archive and re-use. When using `apptainer`, that would be a `“ .sif ”` file.

They can emulate an entire operating system, which also give flexibility regarding what is actually installed into the container. So unlike `conda` (where you are typically restricted to install software from their very extensive catalog), you can set up a container just like an actual computer. A container can mirror any operating system of choice, and it is possible to create and “install” any custom scripts that may be required.

And a lot of software is already provided in existing container that are ready to use. You can browse through them on [dockerhub](#) or [quay](#). In this session we are going to cover how to use existing container only, which still provides the benefits of being able to archive and share the used working environment. The creation of custom containers is a little beyond the scope of this workshop and I refer to the documentation of [apptainer](#) and [buildah](#) for those that want to learn more about the topic.

2.1 In Use

Running software installed within a container is quite similar to running software within a `conda` environment. Again, we need to prefix the command with a call to `apptainer` and to specify which container to use (this can be a url or the path to a `.sif` file on the computer).

In the examples below, we are going to make use of a demo container. This container that includes a simple custom script that prints the content of a file and adds a small header and footer section. The sole purpose of this is to provide a “program” (`dummy_script`) that is not installed on your computer and that thus needs to be run through the container.

So, lets use `dummy_script` to print out the `yml` file that we created earlier:

```
apptainer run docker://khench/dummy_script:v0.2 dummy_script yolo.yml
```

```
=====
||   file content   ||
=====
# conda env create -f yolo.yml
name: yolo
channels:
  - defaults
  - conda-forge
dependencies:
  - python
  - r-base
=====
```

Hopefully you should recognize the content of the `yml` and realize that some lines were tagged in front and behind the actual file content.

2.2 Import and Inspection

As mentioned, a container resembles an actual operating system, and it is possible to inspect the content of a container and to work interactively inside it.

To enter a container we need to call `apptainer shell`:

```
apptainer shell docker://khench/dummy_script:v0.2
```

This will open a shell inside the container, where we can now take a look around.

```
Apptainer> which dummy_script
/usr/local/bin/dummy_script
```

We can use this to inspect the container content. In our example we can take a look at the code behind the bash script `dummy_script`:

```
Apptainer> cat /usr/local/bin/dummy_script
#!/usr/bin/env bash
echo =====
echo ||   file content   ||
echo =====
cat ${1}
echo =====
```

The ability to enter container is a great way to inspect what is going on when using a container and to troubleshoot issues (for example if files can not be seen from inside the container or similar configuration hick-ups).

To leave the container, we can close the shell by typing `exit`.

```
Apptainer> exit
cd $HOME/environment_demo/
```


Finally, if we want to save a container to a file, for offline usage, archiving or sharing, we can use `apptainer pull`. This will create a `.sif` file for the container that can be used instead of the url.

```
apptainer pull docker://khench/dummy_script:v0.2
ls dummy*
dummy_script_v0.2.sif
```

3. WORKFLOW INTEGRATION

Both `conda` environments and containers nicely integrate into the process when working with a workflow manager. In the case of `snakemake` it is possible to define a specific environment or container for each rule of the workflow.

We are going to demonstrate how to do this with a minimal example that runs the tests above, while automatically assigning the correct working environment to each test. Once more, you can import the required code from the `external_resources` folder in your `de.NBI $HOME` directory:

```
rm -r results
cp -r $HOME/external_resources/scripts/workflow code/
cd code
```

When looking at the imported `snakefile`, we see that it makes use of two more directives for the rules: `conda` or `container`.

```
21 rule run_r3:
22     output:
23         png = "../results/img/plot_r3.png"
24     conda: "envs/r3_tidy.yml"
25     shell:
26         """
27         Rscript R/script3.R
28         """
```

When managing the working environments with `snakemake` it is not necessary to call `conda run` or `apptainer run` yourself – `snakemake` will take care of that.

```
49 rule container_check:
50     input:
51         tsv = "../data/penguins.tsv"
52     output:
53         txt = "../results/dummy_test.txt",
54         intermediate = temp("tmp.txt")
55     container: "docker://khench/dummy_script:v0.2"
56     shell:
57         """
58         head {input.tsv} > tmp.txt
59         dummy_script tmp.txt > {output.txt}
60         """
```

First, we are going to do a small dry run to check what to expect when executing the workflow. As you can see, every script should be run once, hopefully using the correct environment.

```
snakemake -n

Job stats:
job              count
-----
all              1
container_check  1
run_py2         1
run_py3         1
run_r3          1
run_r4          1
```

Then, to use `snakemake` in combination with `conda` and `apptainer`, we will need to use the flags `--use-conda`, or `--use-singularity` respectively (`singularity` is the predecessor of `apptainer`). It may be necessary to also use `--singularity-args` to **bind directories into the container** while running the code (don't worry if this is gibberish to you – if you need to use this feature, it will hopefully make sense [🔗](#)).

```
snakemake -c 1 --use-conda --use-singularity --singularity-args "--bind $HOME/  
environment_demo"
```

Hopefully, the workflow should run and produce the following output files:

```
tree ../
```

```
├── code  
│   ├── R  
│   │   ├── script3.R  
│   │   └── script4.R  
│   ├── py  
│   │   ├── demo2.py  
│   │   └── demo3.py  
│   └── workflow  
│       ├── envs  
│       │   ├── py2_base.yml  
│       │   ├── py3_base.yml  
│       │   ├── r3_tidy.yml  
│       │   ├── r4_tidy.yml  
│       │   ├── reproducible.yml  
│       │   └── yolo.yml  
│       └── snakefile  
├── data  
│   └── penguins.tsv  
├── environment_demo.Rproj  
├── results  
│   ├── dummy_test.txt  
│   ├── img  
│   │   ├── plot_r3.png  
│   │   └── plot_r4.png  
│   ├── py2.log  
│   └── py3.log
```